

Week 4 Module:

Date: 09/07/2026

EASY LEVEL:

Problem Statement:

Given a positive integer **N**, determine the total number of digits present in the number.

Your task is to write a program that counts and prints the total number of digits.

Input Format

A single integer **N**.

Output Format

Print a single integer representing the total number of digits in **N**.

Constraints

$$1 \leq N \leq 10^{18}$$

Example 1

Input

12345

Output

5

Explanation

The number **12345** contains 5 digits.

Example 2

Input

7

Output

1

Explanation

The number **7** consists of only one digit.

Week 4 module

DELTA

Date / /

Pg No.

Easy Level

1) Total Number of Digits

- The objective is to determine the total number of digits present in a given positive integer N .
- The input is a single integer N , and the output should be a single integer representing the count of its digits.
- We can repeatedly divide the number by 10. Each division removes the last digit of the number. We keep a counter that increments with each division until the number becomes 0.

code

```
#include <stdio.h>
int main() {
    unsigned long long n;
    int Count = 0;
    scanf("%llu", &n);
```

```
    if (n == 0) {
        Count = 1;
    } else {
        while (n > 0) {
            Count++;
            n /= 10;
        }
    }
```

```
    printf("%d\n", Count);
    return 0;
}
```

I/p:- 12345

O/p:- 5



main.c

```
1  #include <stdio.h>
2
3  int main() {
4      unsigned long long n;
5      int count = 0;
6
7      scanf("%llu", &n);
8
9      if (n == 0) {
10         count = 1;
11     } else {
12         while (n > 0) {
13             count++;
14             n /= 10;
15         }
16     }
17
18     printf("%d\n", count);
19     return 0;
20 }
```



12345

5

...Program finished with exit code 0
Press ENTER to exit console.

Problem Statement

Given a positive integer **N**, reverse its digits and print the reversed number.

Your task is to write a program that reverses the given number.

Input Format

A single integer **N**.

Output Format

Print the reversed number.

Constraints

$$1 \leq N \leq 10^{18}$$

Example 1**Input**

12345

Output

54321

Explanation

Reversing the digits of **12345** gives **54321**.

Example 2**Input**

90010

Output

1009

Explanation

After reversing the digits, the leading zeros are removed automatically, resulting in **1009**.

2. Reverse the Digits

- The program must reverse the digits of a given positive integer N and print the result.
- Any leading zeros that occur after reversing are removed automatically.
- Use the modulo operator ($\% 10$) to extract the last digit. Multiply a tracking reverse variable by 10 and add the extracted digit. Then divide the original number by 10 to move to the next digit.

Code

```
#include <stdio.h>
int main() {
    unsigned long long n, reversed = 0;

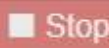
    // Read the input
    scanf("%llu", &n);

    while (n > 0) {
        int remainder = n % 10;
        reversed = reversed * 10 + remainder;
        n /= 10;
    }

    // print the output
    printf("%llu\n", reversed);
    return 0;
}
```

I/p: — 12345

O/p: — 54321



main.c

```
1  #include <stdio.h>
2
3  int main() {
4      unsigned long long n, reversed = 0;
5
6      // Read the input
7      scanf("%llu", &n);
8
9      while (n > 0) {
10         int remainder = n % 10;
11         reversed = reversed * 10 + remainder;
12         n /= 10;
13     }
14
15     // Print the output
16     printf("%llu\n", reversed);
17     return 0;
18 }
```



12345

54321

...Program finished with exit code 0

Press ENTER to exit console.

Problem Statement

Given a positive integer **N**, calculate the sum of all its digits.

Your task is to write a program that computes and prints the sum of the digits.

Input Format

A single integer **N**.

Output Format

Print a single integer representing the sum of all digits.

Constraints

$$1 \leq N \leq 10^{18}$$

Example 1**Input**

12345

Output

15

Explanation

The sum of the digits is:

$$1 + 2 + 3 + 4 + 5 = 15$$

Example 2**Input**

9087

Output

24

Explanation

The sum of the digits is:

$$9 + 0 + 8 + 7 = 24$$

3) Sum of All Digits

→ The task is to compute and print the sum of all the digits in a positive integer N .

→ Similar to extracting digits for reversing, we isolate the last digit using modulo 10 ($n \% 10$). We then add this extracted digit to an ongoing sum variable and reduce the number by dividing it by 10 .

Code

```
#include <stdio.h>
int main()
{
    unsigned long long n, sum = 0;
```

```
// Read the input
    scanf("%llu", &n);
```

```
    while (n > 0)
    {
        sum += (n % 10);
        n /= 10;
    }
```

I/p :- 12345

O/p :- 15

```
// Print the output
    printf("%llu", sum);
    return 0;
```

3



Run



Debug



Stop



Share



Save



{ } Beautify



main.c

```
1  #include <stdio.h>
2
3  int main() {
4      unsigned long long n, sum = 0;
5
6      // Read the input
7      scanf("%llu", &n);
8
9      while (n > 0) {
10         sum += (n % 10);
11         n /= 10;
12     }
13
14     // Print the output
15     printf("%llu\n", sum);
16     return 0;
17 }
```



12345

15

...Program finished with exit code 0
Press ENTER to exit console.

Problem Statement

Given a positive integer **N**, count how many **even** digits and how many **odd** digits are present in the number.

Your task is to write a program that prints the total count of even digits followed by the total count of odd digits.

Input Format

A single integer **N**.

Output Format

Print two integers separated by a space:

- Number of even digits
- Number of odd digits

Constraints

$$1 \leq N \leq 10^{18}$$

Example 1

Input

123456

Output

3 3

Explanation

Even digits are **2, 4, 6**.

Odd digits are **1, 3, 5**.

Therefore, the output is **3 3**.

Example 2

Input

908172

Output

3 3

Explanation

Even digits are **0, 8, 2**.

Odd digits are **9, 1, 7**.

Therefore, the output is **3 3**.

4) Count Even and odd digits

- This problem requires counting the total number of even digits and odd digits in the number N .
- The required output format is two integers separated by a space. The number of even digits followed by the number of odd digits.
- Extract each digit using modulo 10. Use an if/else statement to check if the digit modulo 2 equals 0. If it does, increment the even counter, otherwise, increment the odd counter.

Code

```
#include <stdio.h>
int main () {
    unsigned long long n;
    int even-count = 0, odd-count = 0;
```

```
// Read the input
scanf("%llu", &n);
```

```
if (n == 0) {
    even-count = 1;
}
```

```
while (n > 0) {
    int digit = n % 10;
    if (digit % 2 == 0) {
        even-count++;
    } else {
```

```
        odd-count++;
    }
```

```
n /= 10;
```

```
}
```

```
// Print the output (Even count followed by odd count)
```

```
printf("%d %d\n", even-count, odd-count);
return 0;
```

```
}
```

I/p: 123456

O/p: 3 3

Problem Statement

Given a positive integer **N**, find and print the largest digit present in the number.

Your task is to write a program that determines the maximum digit.

Input Format

A single integer **N**.

Output Format

Print the largest digit present in the given number.

Constraints

$$1 \leq N \leq 10^{18}$$

Example 1**Input**

472953

Output

9

Explanation

The digits are **4, 7, 2, 9, 5, 3**.

The largest digit among them is **9**.

Example 2**Input**

100248

Output

8

Explanation

The digits are **1, 0, 0, 2, 4, 8**.

The largest digit is **8**.

5) Largest Digit

- The task is to find and print the maximum digit present in the positive integer N.
- Initialize a max_digit variable to 0 (since digits range from 0 to 9). As you extract each digit using a while loop, compare it against max_digit. If the extracted digit is greater, update max_digit.

Code

```
#include <stdio.h>
int main() {
    unsigned long long n;
    int max_digit = 0;
```

```
// Read the input
scanf("%llu", &n);
```

I/p:- 472953

O/p:- 9

```
while (n > 0) {
    int digit = n % 10;
    if (digit > max_digit)
        max_digit = digit;
    n /= 10;
}
```

```
printf("%d\n", max_digit);
return 0;
```

```
}
```

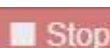
MEDIUM Level

6) Count prime numbers in a range

- The program must count the numbers of prime numbers in the inclusive range $[L, R]$.
- The prime number is defined as a Natural number greater than 1 that has exactly two distinct positive divisors 1 and itself.
- The maximum limit for R is 10^5 , meaning standard int types are sufficient.
- Iterate through all numbers from L to R . for each number, verify if it is prime by checking for factors up to its square root. skip the number 1, as it is not prime by definition.



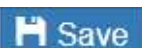
Debug



Stop



Share



Save



{ } Beautify



Download



main.c

```
1  #include <stdio.h>
2
3  int main() {
4      unsigned long long n;
5      int max_digit = 0;
6
7      // Read the input
8      scanf("%llu", &n);
9
10     while (n > 0) {
11         int digit = n % 10;
12         if (digit > max_digit) {
13             max_digit = digit;
14         }
15         n /= 10;
16     }
17
18     // Print the output
19     printf("%d\n", max_digit);
20     return 0;
21 }
```



472953

9

...Program finished with exit code 0
Press ENTER to exit console.

MEDIUM LEVEL:

Problem Statement

Given two positive integers **L** and **R**, count how many prime numbers are present in the inclusive range **[L, R]**.

A **prime number** is a natural number greater than **1** that has exactly two distinct positive divisors: **1** and itself.

Your task is to write a program that counts and prints the total number of prime numbers in the given range.

Input Format

Two integers **L** and **R** separated by a space.

Output Format

Print a single integer representing the total number of prime numbers in the given range.

Constraints

$$1 \leq L \leq R \leq 10^5$$

Example 1

Input

1 10

Output

4

Explanation

The prime numbers between **1** and **10** are: 2, 3, 5, 7

Hence, the total count is **4**.

Example 2

Input

11 20

Output

4

Explanation

The prime numbers between **11** and **20** are: 11, 13, 17, 19

Hence, the total count is **4**.

Code

```
#include <stdio.h>
```

```
// Helper function to check if a number is prime
```

```
int is_prime(int num) {
```

```
    if (num <= 1) return 0;
```

```
    if (num == 2 || num == 3) return 1;
```

```
    if (num % 2 == 0 || num % 3 == 0) return 0;
```

```
    for (int i = 5; i * i <= num; i += 6) {
```

```
        if (num % i == 0 || num % (i + 2) == 0) {
```

```
            return 0;
```

```
        }
    }
```

```
    return 1;
```

```
}
```

```
int main () {
```

```
    int L, R, prime_Count = 0;
```

I/p: 1 10

```
// Read the input
```

```
scanf("%d %d", &L, &R);
```

O/p: 4

```
for (int i = L; i <= R; i++) {
```

```
    if (is_prime(i)) {
```

```
        prime_Count++;
```

```
    }
```

```
// Print the output
```

```
printf("%d\n", prime_Count);
```

```
return 0;
```

```
}
```

7) Sum of proper Divisors

- The objective is to calculate and print the sum of all proper divisors of N .
- A proper divisor is a positive divisor of a number that excludes the number itself.
- N can be up to 10^6 .
- we can find all divisors efficiently by looping up to the square root of N . For every divisor i found, we add i and if N/i is distinct and not equal to N , we add N/i to our sum. Note that 1 is always a proper divisor for numbers greater than 1.



```
1  #include <stdio.h>
2
3  // Helper function to check if a number is prime
4  int is_prime(int num) {
5      if (num <= 1) return 0;
6      if (num == 2 || num == 3) return 1;
7      if (num % 2 == 0 || num % 3 == 0) return 0;
8
9      for (int i = 5; i * i <= num; i += 6) {
10         if (num % i == 0 || num % (i + 2) == 0) {
11             return 0;
12         }
13     }
14     return 1;
15 }
16
17 int main() {
18     int L, R, prime_count = 0;
19
20     // Read the input
21     scanf("%d %d", &L, &R);
22
23     for (int i = L; i <= R; i++) {
24         if (is_prime(i)) {
25             prime_count++;
26         }
27     }
28
29     // Print the output
30     printf("%d\n", prime_count);
31     return 0;
32 }
```



```
10     (n % 2 == 0 || n % (1 + 2) == 0) {  
11         return 0;  
12     }  
13 }  
14 return 1;  
15 }  
16  
17 int main() {  
18     int L, R, prime_count = 0;  
19  
20     // Read the input  
21     scanf("%d %d", &L, &R);  
22
```



1 10

4

...Program finished with exit code 0
Press ENTER to exit console.

Problem Statement

Given a positive integer **N**, find the sum of all its **proper divisors**.

A **proper divisor** of a number is a positive divisor of the number excluding the number itself.

Your task is to write a program that calculates and prints the sum of all proper divisors of **N**.

Input Format

A single integer **N**.

Output Format

Print a single integer representing the sum of all proper divisors of **N**.

Constraints

$$1 \leq N \leq 10^6$$

Example 1

Input

12

Output

16

Explanation

The proper divisors of **12** are:

1, 2, 3, 4, 6

Their sum is:

$$1 + 2 + 3 + 4 + 6 = 16$$

Example 2

Input

15

Output

9

Explanation

The proper divisors of **15** are:

1, 3, 5

Their sum is:

$$1 + 3 + 5 = 9$$

Code

```
#include <stdio.h>
int main () {
    int n;
    long long sum = 0;

    // Read the input
    scanf("%d", &n);

    if (n > 1) {
        sum = 1; // 1 is always a proper divisor
        for (int i = 2; i <= n; i++) {
            if (n % i == 0) {
                sum += i;
                if (i != (n / i)) {
                    sum += (n / i);
                }
            }
        }

        // print the output
        printf("%lld\n", sum);
        return 0;
    }
}
```

I/P:- 12

O/P:- 16

8) Print all factors

DELTA
Date / /
Pg. No.

- The goal is to print all factors of N in ascending order, separated by spaces.
- A factor is defined as an integer that divides the number completely without leaving a remainder.
- N goes up to 10^6 .
- Since we need them printed in perfect ascending order, the simplest and most robust way for $N \leq 10^6$ is to run a single for loop from 1 to N . we check if $N \% i == 0$ and print i immediately. This ensures strict ascending output.



Run Debug Stop Share Save {} Beautify

main.c

```
1  #include <stdio.h>
2
3  int main() {
4      int n;
5      long long sum = 0;
6
7      // Read the input
8      scanf("%d", &n);
9
10     if (n > 1) {
11         sum = 1; // 1 is always a proper divisor
12         for (int i = 2; i * i <= n; i++) {
13             if (n % i == 0) {
14                 sum += i;
15                 if (i != (n / i)) {
16                     sum += (n / i);
17                 }
18             }
19         }
20     }
21
22     // Print the output
23     printf("%lld\n", sum);
24     return 0;
25 }
```



12

16

...Program finished with exit code 0
Press ENTER to exit console.

Problem Statement

Given a positive integer **N**, print all the factors of the number in **ascending order**.

A factor of a number is an integer that divides the number completely without leaving any remainder.

Your task is to write a program that prints all the factors of **N**.

Input Format

A single integer **N**.

Output Format

Print all the factors of **N** in ascending order, separated by spaces.

Constraints

$$1 \leq N \leq 10^6$$

Example 1

Input

12

Output

1 2 3 4 6 12

Explanation

The numbers **1, 2, 3, 4, 6**, and **12** divide **12** exactly without leaving any remainder.

Example 2

Input

25

Output

1 5 25

Explanation

The numbers **1, 5**, and **25** are the only factors of **25**.

Code

```
#include <stdio.h>
int main () {
    int n;

    // Read the input
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        if (n % i == 0) {
            printf("%d ", i);
            if (i != n) {
                printf("\n"); // print space separator
            }
        }
    }

    // print final newline for clean output
    printf("\n");
    return 0;
}
```

I/p: — 12

O/p: — 1 2 3 4 6 12



Run

Debug

Stop

Share

Save

Beautify

main.c

```
1  #include <stdio.h>
2
3  int main() {
4      int n;
5
6      // Read the input
7      scanf("%d", &n);
8
9      for (int i = 1; i <= n; i++) {
10         if (n % i == 0) {
11             printf("%d", i);
12             if (i != n) {
13                 printf(" "); // Print space separator
14             }
15         }
16     }
17
18     // Print final newline for clean output
19     printf("\n");
20     return 0;
21 }
```

12
1 2 3 4 6 12

...Program finished with exit code 0
Press ENTER to exit console.